

SituationReport

testdata_generator

User Manual

Generate synthetic Jira data and export real data from Jira Cloud

situation-report -- github.com/Jaegerfeld/situation-report

For Agile Coaches and PI Managers -- Version 0.9.4

1 What is the Testdata Generator?

The **Testdata Generator** creates synthetic Jira issue data in Jira REST API format. The generated files can be processed directly by **transform_data** — no real Jira instance required.

The generator simulates realistic workflow histories: issues traverse the defined stages, spend varying amounts of time in each stage, occasionally flow backwards, and close at a configurable rate.

Typical use cases:

- **Test a new installation:** Verify that `transform_data` and `build_reports` work correctly
- **Demos and presentations:** Realistic data without privacy concerns
- **Training:** Controlled learning environment with known data
- **Development:** Reproducible test data via the seed parameter

The Testdata Generator complements the **Get Data** module (still in development), which loads real data directly from Jira. For production environments with real Jira data, see Chapter 6.

2 Prerequisites and Start

2.1 Portable package (recommended)

The portable package includes Python already bundled — no separate installation required.

Operating system	Start file
Windows	TestdataGenerator.bat (double-click)
macOS	TestdataGenerator.command (right-click → Open)
Linux	./TestdataGenerator.sh

Note for Windows: If double-clicking shows an error about Python not being found, the ZIP file may have been blocked by Windows. Fix: right-click the ZIP → Properties → Security → Unblock → OK → extract again.

2.2 From source code (Python required)

```
python -m testdata_generator
```

2.3 The user interface

After launch, the main window shows input fields for all parameters. Only the workflow file is required — all other fields have sensible defaults.

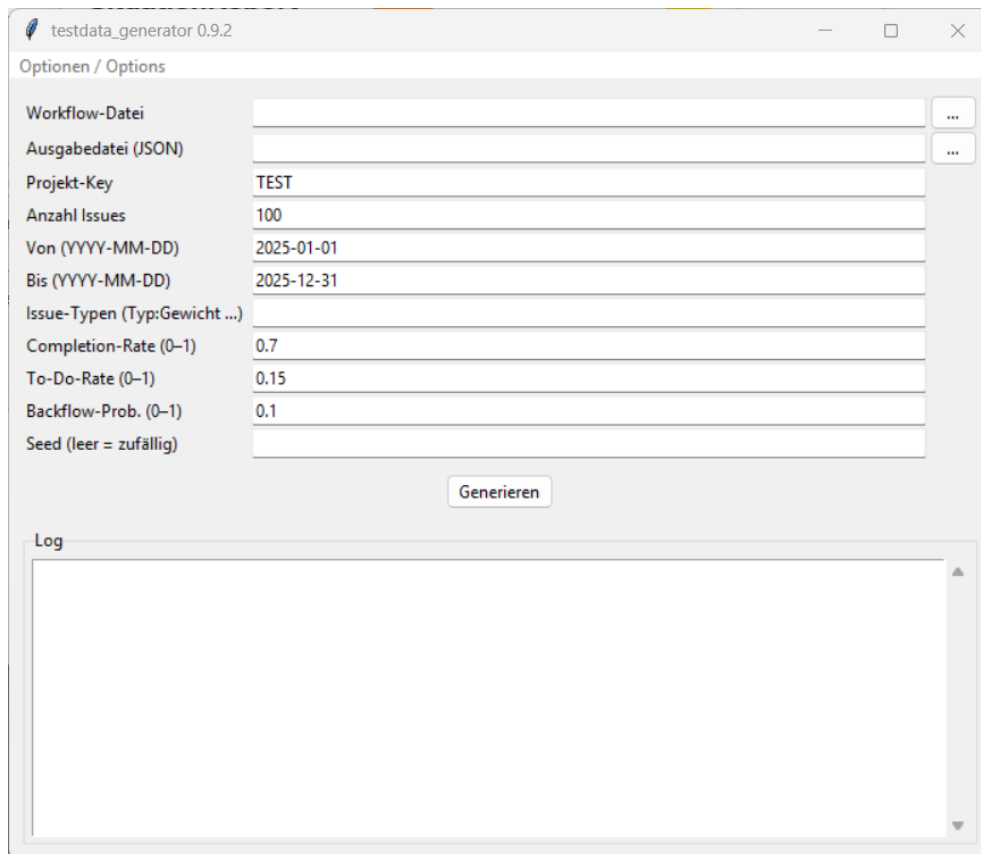


Fig. 1: Testdata Generator user interface

3 The Workflow File

The workflow file defines the stages of your Jira board and is the only required parameter. It uses the same format as `transform_data` — one file works for both modules.

3.1 Format

Each line describes a stage. Aliases (alternative status names in the Jira export) are separated by colons. Two special lines mark the start and end of active work:

```
CanonicalName:Alias1:Alias2
<First>FirstActiveStage
<Closed>ClosedStage
```

Line	Meaning
CanonicalName:Alias1:Alias2	Stage with one or more alternative Jira status names
StageName (no :)	Stage without aliases
<First>StageName	First active stage — where processing begins (cycle time starts here)
<Closed>StageName	Closed stage — marks an issue as completed

3.2 ART_A example workflow

The bundled example workflow `ART_A.txt` models a typical SAFe ART workflow:

```
Funnel:New:Open:To Do
Analysis:In Analysis:Estimated
Program Backlog:Ready for Dev:Backlog
Implementation:In Implementation:In Review:In Progress
Blocker
Validating on Staging:QA
Deploying to Production:Ready for Development
Releasing:Completed
Done:Canceled
<First>Analysis
<Closed>Releasing
```

Notes: 'Funnel' has four aliases — all four Jira status names are normalised to 'Funnel'. Analysis is the first active stage (cycle time begins here). Releasing is the closed stage. Done (after Releasing) also captures Canceled issues.

4 Parameter Reference

All parameters are available both in the GUI and on the command line.

Parameter	Default	Description
--workflow FILE	(required)	Workflow definition file (.txt)
--output FILE.json	<project>_generated.json	Output file path
--project KEY	TEST	Jira project key for generated issues
--issues N	100	Number of issues to generate
--from-date YYYY-MM-DD	2025-01-01	Earliest creation date
--to-date YYYY-MM-DD	2025-12-31	Latest transition date
--issue-types TYPE:W ...	Feature:0.6 Bug:0.3 Enabled:0.1	Issue types with weights (sum need not equal 1)
--completion-rate FLOAT	0.7	Fraction of issues reaching the closed stage (0–1)
--todo-rate FLOAT	0.15	Fraction of open issues staying in early stages (0–1)
--backflow-prob FLOAT	0.1	Probability of a backward transition at each step (0–1)
--seed INT	(random)	Seed for reproducible output (optional)

Tip: Reproducible results

With a fixed seed (e.g. `--seed 42`), the generator always produces the same JSON file for the same parameters — ideal for demos where all participants should see identical results.

5 ART_A – Complete Example

This example shows the full pipeline from generator to finished report using the fictional ART_A project.

Step 1: Generate test data

```
python -m testdata_generator \  
    --workflow testdata_generator/workflow_ART_A.txt \  
    --project ART_A \  
    --issues 200 \  
    --from-date 2025-01-01 \  
    --to-date 2025-12-31 \  
    --issue-types Feature:0.6 Bug:0.2 Enabler:0.2 \  
    --completion-rate 0.75 \  
    --backflow-prob 0.08 \  
    --seed 42 \  
    --output ART_A_generated.json
```

Result: ART_A_generated.json with 200 issues — approximately 150 closed and 50 open across various stages. Seed 42 makes the output reproducible at any time.

Step 2: Process with transform_data

```
python -m transform_data ART_A_generated.json \  
    --workflow testdata_generator/workflow_ART_A.txt
```

Produces three Excel files: ART_A_generated_IssueTimes.xlsx, ART_A_generated_CFD.xlsx, and ART_A_generated_Transitions.xlsx.

Step 3: Create report with build_reports

```
python -m build_reports
```

Load the three Excel files from Step 2 in build_reports — the module generates flow diagrams, cycle time analyses, and metrics.

Expected output (seed 42, 200 issues):

- Issue keys: ART_A-0001 to ART_A-0200
- Issue types: ~120 Feature, ~40 Bug, ~40 Enabler
- ~150 issues with status 'Releasing' or 'Done'
- Creation dates distributed across 2025
- Occasional backward transitions (backflow-prob 0.08 = ~8 %)

6 Exporting Real Data from Jira Cloud

This chapter explains how to export real issue data from Jira Cloud for SituationReport. The export produces the same JSON structure as the Testdata Generator — `transform_data` processes both identically.

6.1 What you need

Requirement	Details
Jira Cloud account	An account with read access to the desired project
API token	Personal token from id.atlassian.com — created once
Project key	The Jira project shortcode, e.g. 'ART_A' or 'SCRUM'
curl or browser	curl for automated queries; browser for quick tests

6.2 Creating an API token

The API token replaces your password for API calls and is created once:

Step	Action
1	Open https://id.atlassian.com in your browser (while logged into Jira)
2	Security → API tokens → Create API token
3	Enter a label, e.g. 'SituationReport', click 'Create'
4	Copy the token and store it safely — it is only shown once!

Security note: Treat the API token like a password. Store it in a password manager and do not share it.

6.3 Querying the Jira REST API

The Jira REST API v2 returns issues as JSON. The parameter `expand=changelog` is **required** — without it, status transitions are missing and `transform_data` cannot compute time-in-stage values.

```
curl -u "name@company.com:YourAPIToken" \
  "https://company.atlassian.net/rest/api/2/search?\
jql=project=ART_A+ORDER+BY+created+ASC\
&expand=changelog\
&maxResults=1000\
&fields=issuetype,created,status,project,summary,resolution" \
  -o ART_A_page1.json
```

Replace `company` with your Atlassian subdomain and `ART_A` with your project key.

Browser method (without curl)

If curl is not installed, open the URL directly in your browser — it uses the existing Jira session:

```
https://company.atlassian.net/rest/api/2/search
?jql=project=ART_A&expand=changelog&maxResults=1000
```

The browser displays the JSON response as text. **Ctrl+S** → file type 'All files' → save as ART_A_page1.json.

6.4 Fields required by transform_data

Jira field	Required	Purpose
key	✓	Issue identifier (e.g. ART_A-001)
fields.issuetype.name	✓	Issue type for filtering (Feature, Bug, ...)
fields.created	✓	Issue creation date
fields.status.name	✓	Current status
changelog (expand=changelog)	✓	Status transitions with timestamps — MUST be set in the URL
fields.resolution	–	Optional — resolution type (e.g. 'Done', 'Won't Fix')
fields.summary	–	Optional — issue title

6.5 Pagination – more than 1,000 issues

Jira returns at most 1,000 issues per request. For larger projects, fetch multiple pages and then merge them with the Helper:

```
# Page 1 (issues 1-1000)
curl -u "name@company.com:Token" \
  "https://company.atlassian.net/rest/api/2/search?jql=project=ART_A\
&expand=changelog&maxResults=1000&startAt=0" \
  -o ART_A_page1.json

# Page 2 (issues 1001-2000)
curl -u "name@company.com:Token" \
  "https://company.atlassian.net/rest/api/2/search?jql=project=ART_A\
&expand=changelog&maxResults=1000&startAt=1000" \
  -o ART_A_page2.json

# Merge files with the Helper
python -m helper ART_A_page1.json ART_A_page2.json \
  --output ART_A_merged.json
```

Increment startAt by 1,000 until the response contains fewer than 1,000 issues — then you have fetched all pages. The **Helper** merges the pages and removes duplicates automatically.

6.6 From JSON export to workflow file

After exporting, create a workflow file that maps your actual Jira statuses. The following Python snippet extracts all status names from the export:

```
import json
data = json.loads(open('ART_A_page1.json').read())
statuses = set()
```



```
for issue in data['issues']:
    for h in issue['changelog']['histories']:
        for item in h['items']:
            if item['field'] == 'status':
                statuses.add(item['toString'])
print(sorted(statuses))
```

From the status names found, create a `workflow.txt` — arrange the statuses in logical order and set the `<First>` and `<Closed>` markers (see Chapter 3).

7 Frequently Asked Questions (FAQ)

What happens if `expand=changelog` is missing?

`transform_data` finds no status transitions and outputs empty time values for all issues. The Excel files contain no usable data. Always include `expand=changelog` in the URL.

Difference between test data and real data?

For demos and training, test data is the better choice — no privacy concerns, reproducible, and fully controlled. For real analyses, actual data reflects the true workflow state and delivers reliable flow metrics.

Can I merge multiple Jira projects?

Yes — either via JQL (`jql=project in (ART_A, ART_B)`) for a single export, or merge two separate exports with the Helper. Make sure the workflows of both projects are compatible.

How large can the JSON file get?

Rule of thumb: approximately 2–5 KB per issue (depending on changelog length). 1,000 issues $\approx$ 2–5 MB. For very large projects (10,000+ issues), use pagination and the Helper.

I get an 'Unauthorized' error when calling curl.

Check: Is the email address correct? Is the API token correct? Are they separated by a colon? Does your account have read access to the project? Tip: Generate a new API token if the old one was lost or expired.

8 Glossary

Term	Explanation
ART	Agile Release Train — a team-of-teams structure in SAFe
API	Application Programming Interface — programming interface
API token	Security key for accessing the Jira REST API; replaces the password in curl calls
Changelog	Jira log of all status changes for an issue (requires expand=changelog)
CFD	Cumulative Flow Diagram — shows the cumulative issue count per stage over time
Closed stage	The stage that marks an issue as completed (workflow.txt: <First>Closed<First>)
Cycle time	Time from the first active stage (<First>First<First>) to the closed stage
First stage	First active stage where cycle time measurement begins (workflow.txt: <First>First<First>)
Helper	SituationReport module for merging multiple Jira JSON files
Issue	A work item in Jira (Feature, Bug, Enabler, Story, ...)
JQL	Jira Query Language — query language for Jira searches, e.g. project=ART_A
JSON	JavaScript Object Notation — format of Jira API exports
Seed	Starting value for the random generator — same seed always produces the same output
Stage	One step in the workflow (corresponds to a Jira column or status name)
Workflow	Ordered sequence of stages that an issue passes through